

作業ログ

報告者：佐藤涼菜

報告日：6 月 23 日

プロジェクト名：マジカル☆観覧車

1. 進捗サマリー（要約）

- 全体進捗率：85[%]
- マイルストーン達成状況：おおよそ達成（テストフェーズは結合待ち）
- 主要成果：
 - 指揮デバイスの筐体改修を完了し，本体・センサの実装（No.2112）を完了
 - Display.ino に静的 IP 設定を追加し，UDP 受信の通信設定を修正（No.2231）
- 重大課題（影響度：高）：
 - テストフェーズ（No.3241）が通信系との結合待ちのため未着手

2. 作業ログ（詳細トラッキング）

2.1 作業ログ

表 1: 作業ログ

No.	作業項目	開始日	予定完了日	状態	進捗率	依存関係・ブロッカー	コメント
2112	本体・センサの実装	5/24	6/17	完了	100%	無	筐体の改修を完了．スライド式可変抵抗器とタクトスイッチのみを露出させ，ユーザの利便性を向上させた．
2231	LED マトリクス制御	5/23	6/12	完了	100%	無	Display.ino に静的 IP 設定（WiFi.config）を追加し，指揮デバイスからの UDP 受信設定を修正した．
2113	BPM・音量変化機構の実装	5/29	6/1	完了	100%	無	スライド式可変抵抗器の動作確認を実施し完了．

次ページへ続く

No.	作業項目	開始日	予定完了日	状態	進捗率	依存関係・ブロッカー	コメント
3241	LED マトリックスの反映確認	6/17	6/30	未着手	0%	通信系との結合待ち	結合テストの実施待ち.
-	作業ログの記入	6/17	6/23	完了	100%	無	特になし.

2.2 日ごとの作業ログ

表 2: 日ごとの作業ログ

日付	No.	作業項目	時間	開始時の状態	終了時の状態	進捗率	コメント
6/17	2112	本体・センサの実装	2h	進行中	進行中	95%	スライド式可変抵抗器の再はんだ付けが完了し動作を確認. 筐体の改修に着手し, スライド式可変抵抗器とタクトスイッチのみを露出させる加工を開始.
〃	2113	BPM・音量変化機構の実装	0.5h	おおよそ完了	完了	100%	スライド式可変抵抗器の動作確認を実施し完了.
6/19	2112	本体・センサの実装	1.5h	進行中	進行中	98%	筐体の改修作業を継続. 操作部の露出加工と内部配線の整理を実施.
6/20	2112	本体・センサの実装	1h	進行中	完了	100%	筐体の最終仕上げを行い, 本体・センサの実装を完了.
6/23	2231	LED マトリックス制御	1h	完了	完了	100%	Display.ino に静的 IP 設定 (WiFi.config) を追加. 指揮デバイスの送信先 IP アドレスと一致させるよう修正した.
〃	-	作業ログの記入	1h	未着手	完了	100%	特になし.

3. 計画時の GC と現在の GC

計画段階のガントチャート (GC) を図 1 (実装フェーズ) および図 2 (テストフェーズ) に示す.

観覧車型デバイスを用いた楽器間通信演奏システム			5月											6月														7月																						
No.	作業	担当者	22	23	24	25	26	27	28	29	30	31	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	1	2	3	4	5	6	7	
2111	部品機材用意	山口 佐藤																																																
2112	本体・センサの実装	山口 佐藤																																																
2113	BPM・音量変化機構の実装	山口 佐藤																																																
2121	LEDマトリックスの用意	山口 佐藤																																																
2131	部品機材用意	山口 佐藤																																																
2132	3Dプリンタによる観覧車部品用意	山口 佐藤																																																
2133	観覧車回路実装	山口 佐藤																																																
2134	音情報回路実装	山口 佐藤																																																
2135	観覧車(中継機)本体の製作	山口 佐藤																																																
2211	演奏情報の通信実装	梅野 薄井																																																
2212	演奏実装	梅野 薄井																																																
2221	通信開始合図の送信実装	宇井 菊池																																																
2222	BPM・音量情報送信実装	宇井 菊池																																																
2223	音楽・楽器同期の実装	宇井 菊池																																																
2224	観覧車の制御	宇井 菊池																																																
2231	LEDマトリクス制御	山口 佐藤																																																
2241	各楽器の音色の再現	梅野 薄井																																																
2242	演奏情報からの発音機能実装	梅野 薄井																																																

図 1 計画段階のガントチャート (実装フェーズ)

次に、現在の GC を図 3 に示す。達成した項目は緑色で塗られている。

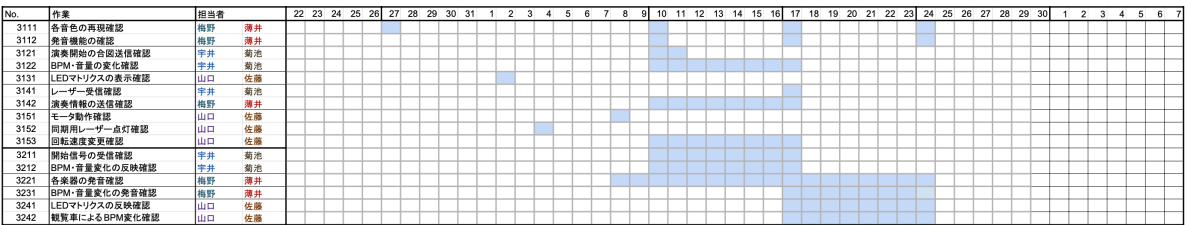


図 2 計画段階のガントチャート（テストフェーズ）

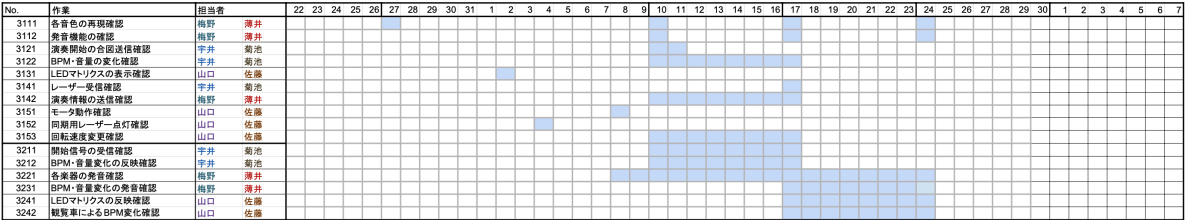


図 3 現在のガントチャート

4. 課題管理

課題	発生原因	影響度	優先度	対策	期限	状態
可変抵抗器がブレッドボード上で安定接続できない(No.2112/2113に影響)	スライド式可変抵抗器の端子形状がブレッドボードに適合しないため、	高	高	スライド式可変抵抗器を基盤に設置し、筐体へ固定した。	6/12	対応済
1 機のスライド式可変抵抗器のはんだ付け設置不良（No.2112に影響）	基盤へのはんだ付け時に接触不良が発生したため、	中	高	6/17 に再はんだ付けを実施し、動作を確認した。	6/17	対応済
テストフェーズが結合待ち (No.3241 に影響)	通信系との結合が完了していないため、LED マトリクスの反映確認テストが実施できない。	高	中	通信系の準備完了を待ち、結合テストを実施する。	6/30	対応中

※影響度＝プロジェクトへのインパクト

※優先度＝対応の緊急性

5. AI 利用状況と検証（再現性重視）

5.1 利用内容

- 利用目的：実装支援
- 入力（プロンプト要旨）：
 - Display.ino において、指揮デバイスからの UDP 受信に必要な静的 IP 設定（WiFi.config）の追加方法について

質問した。

- 出力概要：
 - IPAddress 定数 (localIP, gateway, subnet) の定義方法および, setup() 内で WiFi.begin() の前に WiFi.config() を呼び出す修正方針が得られた。
-

5.2 検証方法

- 検証手法：
 - AI が提示した実装を Display.ino に追加し, Arduino IDE 上でコンパイルが通ることを確認した。
 - AI の出力内容と Arduino 公式ドキュメントを比較し, WiFi.config() の引数仕様に誤りがないか確認した。
 - 参照資料：
 - Arduino 公式ドキュメント：<https://docs.arduino.cc/>
 - 検証手順：
 1. AI へ静的 IP 設定の追加方法について質問した。
 2. AI が提示した内容をもとに IPAddress 定数を定義し, setup() 内に WiFi.config() の呼び出しを追加した。
 3. Arduino IDE でコンパイルし, エラーがないことを確認した。
 4. AI の出力内容を Arduino 公式ドキュメントと比較し, WiFi.config() の使用方法に誤りがないことを確認した。
-

5.3 ファクトチェック結果

- 一致点：
 - AI が提示した WiFi.config(ip, gateway, subnet) の引数順序および IPAddress の定義方法は, Arduino 公式ドキュメントの内容と一致していた。
 - 不一致・疑義：
 - 特になし。
 - 最終判断：
 - 採用
 - 根拠：
 - Arduino 公式ドキュメントとの比較により, AI の出力内容が正確であることを確認したため。
 - Arduino IDE でコンパイルが通り, 実装に問題がないことを確認したため。
-

6. 次回計画

- 目標：通信系との結合テストの実施
 - 通信系の準備が完了次第, LED マトリクスの反映確認テスト (No.3241) を実施する。
 - 指揮デバイスからの UDP 送信により BPM 値が LED マトリクスに正しく表示されることを確認する。
- 達成基準：
 - 指揮デバイスから BPM 値を送信し, 表示デバイスの LED マトリクスに正しく反映されること。
 - 演奏終了ヘッダ ('E') の受信時に LED マトリクスが消灯すること。
- 期限：
 - 2026/6/30

7. 付録

No.2231 (LED マトリクス制御) のプログラムを以下に示す.

7.1 Display.h

```
1  #ifndef DISPLAY_H
2  #define DISPLAY_H
3
4  #include <Arduino.h>
5  #include "Arduino_LED_Matrix.h"
6  #include <WiFiS3.h>
7
8  #define BPM_STEP_1    60
9  #define BPM_STEP_2    90
10 #define BPM_STEP_3    120
11 #define BPM_STEP_4    150
12 #define BPM_STEP_5    180
13
14 #define LOCAL_PORT     2390
15 #define HEADER_BPM     'B'
16 #define HEADER_END     'E'
17
18 #define FONT_WIDTH     3
19 #define FONT_HEIGHT    5
20
21 #define MATRIX_ROWS    8
22 #define MATRIX_COLS    12
23
24 void displaySetup();
25 void checkUDP();
26 void displayBPM(int bpm);
27 void clearDisplay();
28 void drawDigit(int digit, int x_offset);
29
30 extern const uint8_t font3x5[10][15];
31 extern ArduinoLEDMatrix matrix;
32 extern WiFiUDP Udp;
33
34 #endif
```

7.2 Display.ino

```
1  #include "Display.h"
2
3  WiFiUDP Udp;
4
5  const char SSID[] = "";
6  const char PASS[] = "";
7
8  IPAddress localIP(192, 168, 11, 10);
9  IPAddress gateway(192, 168, 11, 1);
10 IPAddress subnet(255, 255, 255, 0);
11
12 void setup() {
13     displaySetup();
14
15     if (SSID[0] != '\0') {
16         WiFi.config(localIP, gateway, subnet);
17         while (WiFi.begin(SSID, PASS) != WL_CONNECTED) {
18             delay(1000);
19         }
20     }
21     Udp.begin(LOCAL_PORT);
22 }
23
24 void loop() {
25     checkUDP();
26 }
```

7.3 Display.cpp

```

1 #include "Display.h"
2 #include "Arduino_LED_Matrix.h"
3
4 const uint8_t font3x5[10][15] = {
5     {1,1,1, 1,0,1, 1,0,1, 1,0,1, 1,1,1}, // 0
6     {0,1,0, 1,1,0, 0,1,0, 0,1,0, 1,1,1}, // 1
7     {1,1,1, 0,0,1, 1,1,1, 1,0,0, 1,1,1}, // 2
8     {1,1,1, 0,0,1, 0,1,1, 0,0,1, 1,1,1}, // 3
9     {1,0,1, 1,0,1, 1,1,1, 0,0,1, 0,0,1}, // 4
10    {1,1,1, 1,0,0, 1,1,1, 0,0,1, 1,1,1}, // 5
11    {1,1,1, 1,0,0, 1,1,1, 1,0,1, 1,1,1}, // 6
12    {1,1,1, 0,0,1, 0,0,1, 0,0,1, 0,0,1}, // 7
13    {1,1,1, 1,0,1, 1,1,1, 1,0,1, 1,1,1}, // 8
14    {1,1,1, 1,0,1, 1,1,1, 0,0,1, 1,1,1} // 9
15 };
16
17 static const int bpmTable[5] = {
18     BPM_STEP_1, BPM_STEP_2, BPM_STEP_3,
19     BPM_STEP_4, BPM_STEP_5
20 };
21
22 ArduinoLEDMatrix matrix;
23
24 void displaySetup() {
25     matrix.begin();
26     uint32_t warmUp[3] = {0, 0, 0};
27     matrix.loadFrame(warmUp);
28     delay(200);
29 }
30
31 static uint8_t frameBuffer[MATRIX_ROWS][MATRIX_COLS];
32 static int currentBPM = -1;
33
34 void checkUDP() {
35     int packetSize = Udp.parsePacket();
36     if (packetSize <= 0) return;
37
38     uint8_t buffer[2] = {0};
39     int len = Udp.read(buffer, sizeof(buffer));
40     if (len < 2) return;
41
42     char header = (char)buffer[0];
43     uint8_t stage = buffer[1];
44
45     if (header == HEADER_END) {
46         clearDisplay();
47         return;
48     }
49
50     if (header != HEADER_BPM) return;
51     if (stage < 1 || stage > 5) return;
52
53     int bpm = bpmTable[stage - 1];
54
55     if (bpm != currentBPM) {
56         currentBPM = bpm;
57         displayBPM(bpm);
58     }
59 }
60
61 void displayBPM(int bpm) {
62     memset(frameBuffer, 0, sizeof(frameBuffer));
63
64     int digits[3];
65     int numDigits;
66
67     if (bpm >= 100) {
68         numDigits = 3;
69         digits[0] = bpm / 100;
70         digits[1] = (bpm / 10) % 10;
71         digits[2] = bpm % 10;
72     } else if (bpm >= 10) {
73         numDigits = 2;
74         digits[0] = bpm / 10;
75         digits[1] = bpm % 10;
76     } else {
77         numDigits = 1;
78         digits[0] = bpm;
79     }
80
81     int totalWidth = numDigits * FONT_WIDTH
82                     + (numDigits - 1) * 1;
83     int startX = (MATRIX_COLS - totalWidth) / 2;
84
85     for (int i = 0; i < numDigits; i++) {
86         drawDigit(digits[i],

```

```

87         startX + i * (FONT_WIDTH + 1));
88     }
89
90     uint32_t bitmap[3] = {0, 0, 0};
91     for (int row = 0; row < MATRIX_ROWS; row++) {
92         for (int col = 0; col < MATRIX_COLS; col++) {
93             if (frameBuffer[row][col]) {
94                 int n = row * MATRIX_COLS + col;
95                 bitmap[n / 32] |=
96                     (1UL << (31 - (n % 32)));
97             }
98         }
99     }
100
101     matrix.loadFrame(bitmap);
102 }
103
104 void drawDigit(int digit, int x_offset) {
105     if (digit < 0 || digit > 9) return;
106
107     const int y_offset =
108         (MATRIX_ROWS - FONT_HEIGHT) / 2;
109
110     for (int y = 0; y < FONT_HEIGHT; y++) {
111         for (int x = 0; x < FONT_WIDTH; x++) {
112             int col = x_offset + x;
113             int row = y_offset + y;
114             if (col >= 0 && col < MATRIX_COLS
115                 && row >= 0 && row < MATRIX_ROWS) {
116                 frameBuffer[row][col] =
117                     font3x5[digit][y * FONT_WIDTH + x];
118             }
119         }
120     }
121 }
122
123 void clearDisplay() {
124     currentBPM = -1;
125     uint32_t blank[3] = {0, 0, 0};
126     matrix.loadFrame(blank);
127 }

```